CS2103 / Lab – Assignment 07

Topic: Informed Search (A*) and Adversarial Search

Instructions: Complete all five questions. Implement **Greedy Best-First Search, A*, Minimax, and Alpha–Beta Pruning from scratch**. Built-in search or game-solving algorithms are not allowed except for verification.

Q1. Greedy Best-First Search and A*

Create a weighted graph with at least 10 states, including a start state, goal state, edge costs, and heuristic values $h(n)$. Implement

$$\text{Greedy Best-First: } f(n) = h(n), \qquad \text{A*: } f(n) = g(n) + h(n).$$

For both algorithms, report the **path found, total path cost, and number of expanded nodes**. Briefly explain why the two methods may follow different paths.

Q2. Effect of Heuristic in A*

Create a 15×15 grid with obstacles and use four-directional unit-cost movement. Implement A* using Manhattan distance,

$$h(n) = |x_n - x_g| + |y_n - y_g|.$$

Run the same problem with $h(n) = 0$. Visualize the final path and compare the **path cost and expanded nodes**. Explain how the heuristic changes the search effort.

Q3. Minimax from Scratch

Implement **Tic-Tac-Toe** using `player(s)`, `actions(s)`, `result(s,a)`, `terminal(s)`, and `utility(s)`.

Use

$$U(s) = \begin{cases} +1, & \text{MAX wins,} \\ 0, & \text{draw,} \\ -1, & \text{MIN wins.} \end{cases}$$

For three non-terminal board states, report the **optimal move, Minimax value, and number of evaluated nodes**.

Q4. Alpha–Beta Pruning

Extend the Minimax implementation using Alpha–Beta Pruning, where α represents the best value found for MAX and β the best value found for MIN.

For the same board states, compare **Minimax and Alpha–Beta** using the optimal move, utility value, evaluated nodes, and number of pruned branches. Verify that pruning does not change the optimal decision.

Q5. Move Ordering and Cutoff Challenge

Run Alpha–Beta Pruning using:

(i) random move ordering,
(ii) promising moves first,
(iii) promising moves last.

Compare the evaluated nodes and pruning operations. Then apply a fixed cutoff depth d and use

$$eval(s) = \sum_{i=1}^{m} w_i f_i(s),$$

with suitable Tic-Tac-Toe features. Briefly explain how **move ordering, cutoff depth, and evaluation-function quality** affect search efficiency and decision quality.

Note: Students must be able to explain $g(n)$, $h(n)$, $f(n)$, Greedy Best-First Search, A*, Minimax, MAX/MIN levels, Alpha–Beta Pruning, move ordering, cutoff, and evaluation functions during viva.

Bonus: Bonus marks will be awarded for the effective use of **Claude Code or VS Code prompts**, provided the prompts used are documented in the submitted notebook.